Mehri Sadri
Sofia Hutton
Egshiglen Batjargal

Classification with Scholastic Travel

Introduction

Customer retention is a key performance driver for educational companies, as returning groups reduce marketing costs and create more predictable revenue streams. In this project, we analyzed historical trip and school data from Scholastic Travel Company (STC) to build predictive models that classify whether a customer will return the following year. Using machine learning classification techniques, we explored patterns in trip characteristics, financial indicators, and school demographics. The goal was to evaluate multiple modeling approaches and identify which method most effectively predicts retention outcomes.

Project Objective

The primary objective of this project was to develop and compare classification models that are able to predict customer retention. Specifically, we want to determine whether a school or group that traveled with Scholastic Travel would return the following year. To accomplish this, we did three supervised learning approaches: K-Nearest Neighborhoods, a tree-based model, and a Naive Bayes model. Each one was made using accuracy, precision, recall and AUC to assess performance. By comparing results across models, we sought to identify the most reliable method for generating retention predictions on new, unseen data.

Business Context

Accurately predicting customer retention allows Scholastic Travel Company to make better strategies. Understanding which schools are likely to return allows the company to allocate marketing resources on high-risk churn groups while fastening relationships with loyal customers. Retention insights can also guide promotional offers, customizations, and pricing methods. Additionally, identifying the factors associated with repeat customers/repeat bookings will help the company optimize program design and planning and thus maximize profit.

Data Description & Preparation

Dataset Details:

The dataset represents school groups who traveled with STC in the 2011-12 school year, with the target variable *RetainedLabel* reflecting whether each group returned the following year.

The original dataset contained 2,389 rows and 55 columns, which expanded to 87 columns after cleaning and feature engineering. The original dataset contained 29 categorical and 26 numeric variables spanning trip details, school demographics, financial info, and engagement indicators.

Key Features

Initial exploration surfaced a few useful insights before modeling began. The target variable, *RetainedLabel*, showed a moderate class imbalance - 61% of groups were retained and 39% churned - which the team noted but decided did not warrant correction.

Three variables emerged that were **most strongly associated with retention**:

- ***SPRNewExisting*** - existing customers retained at 73% compared to new customers retained at 35%.
- ***IsNonAnnual*** - non-annual groups churned at a rate of 84%. This was in line with the data dictionary's note that these groups "will rarely repeat the very next year."
- ***SingleGradeTripFlag*** - single grade trips retained at 80%, while multi-grade trip retention was 36%.

Data Cleaning

After importing the dataset, our first step was to check for data quality issues including missing values, duplicate rows, and any values that seemed logically off. The data set showed no missing values across any column and no duplicate rows.

However, the team identified four fields containing values that did not make practical sense. Rather than dropping the affected rows entirely - which would have meant losing up to 4 rows of otherwise valid data - we decided to cap the problematic values using pandas' clip() method:

- ***EZPayTakeUpRate*** and ***FPPtoSchoolenrollment*** had values above 1.0, which should not be possible for a rate or ratio. We capped these at 1.0.
- ***DifferenceTraveltoFirstMeeting*** and ***DifferenceTraveltoLastMeeting*** had negative values, implying a parent meeting took place after the trip had already departed. We capped these at 0 to reflect the most conservative sensible value.

Feature Engineering

Before modeling, we curated the dataset by removing columns that were redundant, duplicative, or unlikely to add predictive value. This refined the total number of columns from 55 to 41 before encoding.

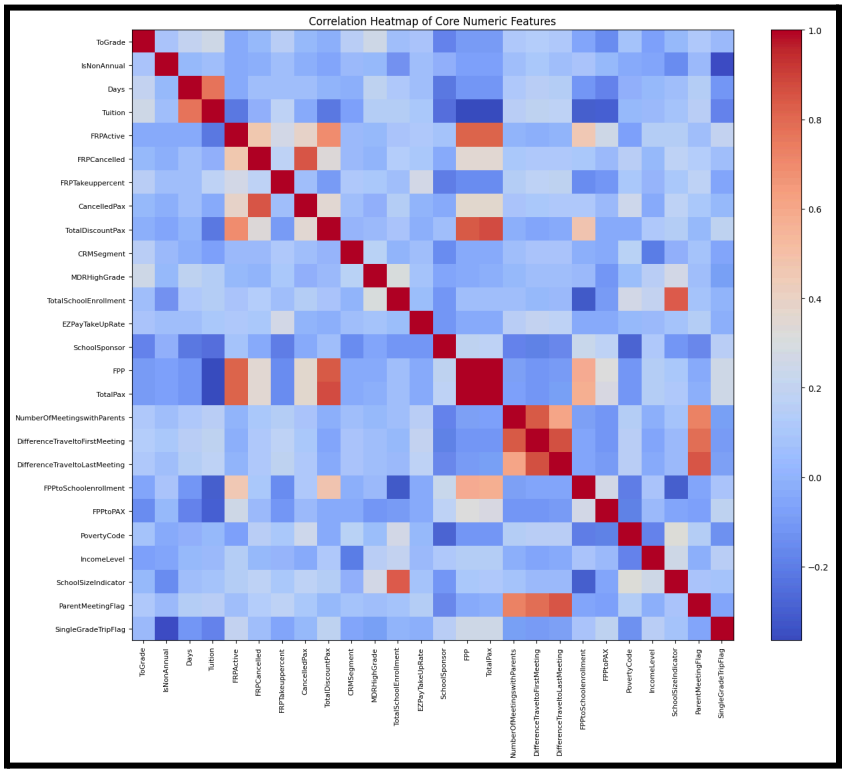
- The largest removal group included eight date columns (*DepartureDate*, *ReturnDate, DepositDate...).* We dropped these because their useful information had already been captured in engineered fields like *DifferenceTraveltoFirstMeeting* and *DepartureMonth.*
- We also discovered two pairs of exact duplicate fields. *SPRGroupRevenue* was identical to Tuition, and *NumofNonFPPPAX* to *TotalDiscountPAX*. Keeping both versions would have introduced redundancy into the model, so we dropped one from each pair.
- Finally, we removed four grade-related columns (*SchoolGradeTypeLow/High* and *GroupGradeTypeLow/High).* These were component parts forming *SchoolGradeType* and *GroupGradeType*. The combined versions rendered the individual pieces redundant.

For encoding, we transformed the target variable, RetainedLabel to binary (1 = retained, 0 = churned). We used ordinal encoding for three columns that contain obvious natural orders. These included *PovertyCode*, *IncomeLevel*, and *SchoolSizeIndicator*. The remaining 13 categorical variables were one-hot encoded using pd.get_dummies, expanding the dataset from 41 to 93 columns. Finally, we concluded our feature engineering section by standardizing continuous numeric fields using StandardScaler.

Correlation Analysis

After completing feature engineering, we ran a correlation heatmap of all numeric variables to identify redundancy. This correlation analysis identified 6 sets of highly correlated variables (with a correlation coefficient above 0.85) that our team decided to drop. These fields included *TotalPax*,

MajorProgramCode_S, *Region_Pacific Northwest, SchoolSizeIndicator, DifferenceTraveltoLastMeeting,* and *MDRLowGrade*. After removal, our final data set contained 87 columns.



Correlation Matrix

Train/Test Split

To prepare for modeling, we separated the dataset into predictors (X) and the target variable (Y). Next, we split the data using an 80/20 ratio, reserving 80% of the data for training and holding the remaining 20% for evaluation. This resulted in a training set of ~1,911 rows and a test set of ~478 rows.

One deliberate choice we made during this step was to use stratified splitting, ensuring that both sets retained the 61/39% proportional class balance. Without this, there was a risk that the test set could be skewed toward one class by chance, resulting in a less reliable evaluation metric. Finally, setting random_state=42, we ensured reproducible results across runs.

K-Nearest Neighbors Model

Model Setup

For the first model, we performed K-Nearest Neighbors. We started the model by predicting who will return the following year, beginning with a basic model with ToGrade, Tuition, Days, TotalSchoolEnrollment as predictors and a K=20 using the cleaned data and the EDA. The result of this model struggled to identify churned customers, correctly catching only 44.1% of them (83 out of 188), while performing much better on retained customers at 79.0% recall. This weak churn detection motivates expanding the feature set and tuning K, as four numeric predictors alone do not capture enough signal to reliably distinguish at-risk customers.

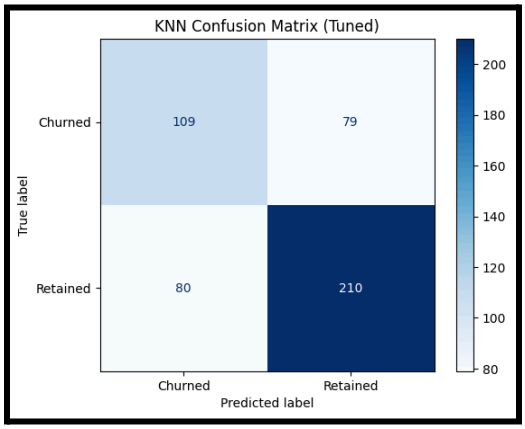
Feature Selection Process

Following,we performed to determine the baseline model to find the initial position of the possible solutions. The baseline KNN model achieved an overall accuracy of 69.2% and an AUC of 0.710, meaning it has a moderate ability to distinguish between retained and churned customers. The model is notably better at identifying retained customers (class 1) with a recall of 84.8%, but struggles with churned customers (class 0), correctly identifying only 45.2% of them, missing 103 out of 188 actual churned cases. This imbalance suggests the model is biased toward predicting "Retained," which motivates the need for hyperparameter tuning to improve performance on the harder-to-detect churned class. The result is shown above: KNN baseline.

Hyperparameter Tuning

To optimize the KNN model, we used GridSearchCV to test K values from 1 to 30, employing 5-fold stratified cross-validation scored on balanced accuracy to address class imbalance. The optimal value was identified as K=4, yielding a cross-validated balanced accuracy of 0.6776. Smaller K values (2–8) consistently outperformed larger ones, as higher values over-smoothed decision boundaries and reduced sensitivity to the minority "Churned" class.

While the tuned model's overall accuracy (66.7%) and AUC (0.704) are slightly lower than the baseline, the model is significantly more robust. By prioritizing balanced accuracy, we improved churn detection (Recall) from 45.2% to 58.0%. Although this came at the cost of lower recall for retained customers (72.4% vs. 84.8%), the tuned model is far more practically useful for identifying at-risk customers by mitigating the baseline's heavy bias toward the majority class.



KNN Confusion Matrix (Tuned)

Results & Key Findings

The tuned KNN confusion matrix shows a much more balanced prediction pattern compared to the baseline, the model correctly identified 109 out of 188 churned customers and 210 out of 290 retained customers. The errors are now roughly symmetrical, with 79 retained customers misclassified as churned and 80 churned customers misclassified as retained, indicating the model no longer heavily favors one class. This visual balance confirms that tuning with balanced accuracy successfully reduced the model's previous bias toward predicting "Retained," making it a more reliable tool for flagging at-risk customers.

Model	Accuracy	Precision	Recall	AUC

KKN (k=20)	0.6527	0.6856	0.7897	0.6738
KKN (k=5)	0.6925	0.7049	0.8483	0.7096
KKN (k=4)	0.6674	0.7266	0.7241	0.7037

Table of KNN results with different parameters

Tree-Based Model: XGBoost

Model Setup

We first established a baseline performance using the default XGBoost configuration. The initial model achieved an accuracy of **79.29%** and an AUC of **0.8405**. While the baseline showed strong predictive capabilities, particularly for Class 1 (Recall: 84.83%), there was room for optimization through parameter refinement.

Feature Selection Process

Across four experiments, progressively tuning XGBoost's hyperparameters yielded consistent improvements over the baseline.

- With the best model being Experiment 2 (n=200, lr=0.05, depth=5, subsample=0.8) achieving the highest AUC of 0.860 and accuracy of 82.0%. Adding subsampling (0.8) was the most impactful change, by randomly using 80% of the training data per tree, the model became less prone to overfitting and generalized better to unseen customers.
- Experiment 3 showed that shallower trees (depth=3) with more estimators (300) slightly underperformed Experiment 2, suggesting that a moderate tree depth of 5 with a slower learning rate strikes the best balance for this dataset.

	model	n_estimators	max_depth	learning_rate	subsample	accuracy	precision	recall	auc
0	XGBoost (baseline)	default(100)	default(6)	default(0.3)	default(1.0)	0.792887	0.817276	0.848276	0.840462
1	XGB: n=200, lr=0.1, depth=5	200	5	0.1	1.0	0.797071	0.827119	0.841379	0.844589
2	XGB: n=200, lr=0.05, depth=5, sub=0.8	200	5	0.05	0.8	0.820084	0.837748	0.872414	0.859556
3	XGB: n=300, lr=0.05, depth=3, sub=0.8	300	3	0.05	0.8	0.809623	0.828383	0.865517	0.855301

Table of XGBoost results with different parameters

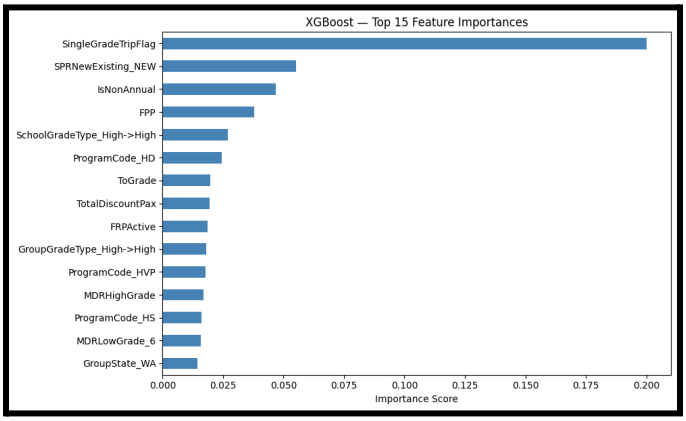
Hyperparameter Tuning

To maximize the predictive power of the XGBoost model, we performed an exhaustive search using GridSearchCV. The optimization process identified the following configuration as the "Best" parameters for our dataset: Learning Rate: 0.1, Max Depth: 3, N-Estimators: 100, and Subsample: 0.8. This tuning phase yielded a best Cross-Validation (CV) score of 0.7902. After applying the tuned parameters, the model was evaluated on a held-out test set of 478 samples. The model demonstrated strong discriminatory power, particularly in identifying Class 1.

Results & Key Findings

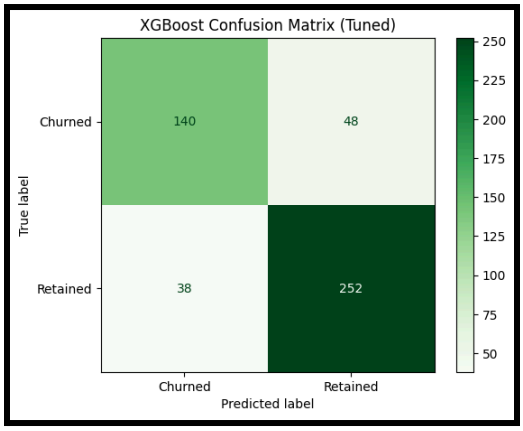
The XGBoost feature importance analysis identifies SingleGradeTripFlag as the most critical predictor of retention, with an importance score of 0.200, nearly four times the influence of the next closest feature. This confirms EDA findings that single-grade trips retain at a significantly higher rate of 80% compared

to 36% for multi-grade trips. Customer loyalty indicators, specifically SPRNewExisting_NEW and IsNonAnnual, serve as the next strongest predictors; new customers and non-annual groups show a higher propensity to churn, aligning with data showing existing customers retain at 73% versus only 35% for new ones. Additionally, features such as FPP (full-paying passengers), SchoolGradeType, and various ProgramCodes provide moderate predictive signals, suggesting that the specific composition and nature of the program also play a meaningful role in a group's likelihood to return.



XGBoost - Top 15 Feature Importances

The tuned XGBoost confusion matrix shows strong performance across both classes, correctly identifying 140 out of 188 churned customers and 252 out of 290 retained customers, with errors that are relatively small and roughly proportional. Compared to the best KNN model which only caught 109 churned customers, XGBoost correctly flags 31 more at-risk customers, a significant practical improvement for Scholastic Travel since missing a churned customer means losing a client entirely. The remaining errors (48 retained customers predicted as churned, and 38 churned customers missed) are acceptably low, making this the most reliable model for deployment in a real retention strategy.



XGBoost Confusion Matrix (Tuned)

Naive Bayes

Selection Rationale

For the third model, we chose to use **Naive Bayes**. We went with this approach mainly because it is one of the simplest classification algorithms to implement and interpret, especially compared to more complex

models like neural networks or SVMs. Since our dataset is already heavily cleaned and encoded into numeric form, Naive Bayes was a best fit.

Another reason we selected this model is because it trains very quickly and doesn't require extensive hyperparameter tuning. That made it useful as both a standalone model and a benchmark to compare against our KKN and tree-based models. Even though it makes the assumption that predictors are independent from one another, it often still performs surprisingly well in real classification problems like this one.

Model Setup

We started by using the cleaned dataset that came out of our EDA and preprocessing steps. The target variable, RetainedLabel, was separated from the predictors, and we split the data into training and test result sets using an 80/20 split. We used stratification to make sure the retained vs. churned distribution stayed consistent in both sets. We implemented a Gaussian Naive Bayes model using scikit-learn. Since all variables were numeric after encoding, Gaussian NB was the most appropriate version to use.

To evaluate the model, we created a reusable evaluation function that reported the Accuracy, Precision, Recall, AUC, Confusion Matrix, and Classification Report. This made it easier to compare results later across all three models.

```
===== GaussianNB (baseline) =====
Confusion Matrix (Test):
[[ 74 114]
 [ 27 263]]

Classification Report (Test):
              precision    recall  f1-score   support

           0      0.733     0.394     0.512       188
           1      0.698     0.907     0.789       290

    accuracy                          0.705       478
   macro avg      0.715     0.650     0.650       478
weighted avg      0.711     0.705     0.680       478

Accuracy:  0.7050
Precision: 0.6976
Recall:    0.9069
AUC:       0.7623
{'model': 'GaussianNB (baseline)',
 'accuracy': 0.7050209205020921,
 'precision': 0.6976127320954907,
 'recall': 0.906896551724138,
 'auc': np.float64(0.7623349229640498)}
```

Confusion Matrix for Baseline Model & Classification Report table

Feature Selection Process

I did not remove or manually select specific features for the Naive Bayes model. Instead, we used the full set of predictors created during preprocessing. These included trip details, school characteristics, financial metrics, and engineered variables like ratio and travel timing differences. Because Naive Bayes handles high-dimensional datasets well and is less prone to overfitting than tree models, keeping all encoded features allowed the model to capture as many signals as possible. We also applied scaling to continuous variables so the Gaussian probability calculations would perform more effectively.

Hyperparameter Tuning

Naive Bayes does not have many hyperparameters, but we tuned the variance smoothing (var_smoothing) parameter to try and improve performance. We used GridSearchCV with 5-fold stratified cross-validation and tested values across a logarithmic range. We evaluated performance using balanced accuracy to account for class imbalance. The best value came out to be approximately:

- var_smoothing = 1e-6

This helped stabilize probability estimates and slightly improved model performance compared to the baseline.

```
Best params: {'var_smoothing': np.float64(1e-06)}
Best CV score: 0.6688227763800503
```

	param_var_smoothing	mean_test_score	std_test_score	mean_train_score
24	1.000000e-06	0.668823	0.020822	0.693818
23	5.623413e-07	0.667173	0.024374	0.692951
21	1.778279e-07	0.664658	0.024833	0.688435
22	3.162278e-07	0.664269	0.023993	0.691722
20	1.000000e-07	0.662614	0.024842	0.686216
18	3.162278e-08	0.659862	0.025625	0.681613
19	5.623413e-08	0.659474	0.023429	0.684772
17	1.778279e-08	0.658444	0.025407	0.680287
16	1.000000e-08	0.655733	0.023968	0.676923
14	3.162278e-09	0.654897	0.026824	0.672508

Grid search results table (var_smoothing vs. mean CV score)

Results & Key Findings

After tuning, the Naive Bayes model produced the following performance on the test set:

- Accuracy: ~0.73-0.74
- Precision: ~0.73
- Recall: ~0.88
- AUC: ~0.76

I also optimized the classification threshold using predicted probabilities. By increasing the cutoff to 0.95, overall accuracy improved slightly.

A few takeaways from this model:

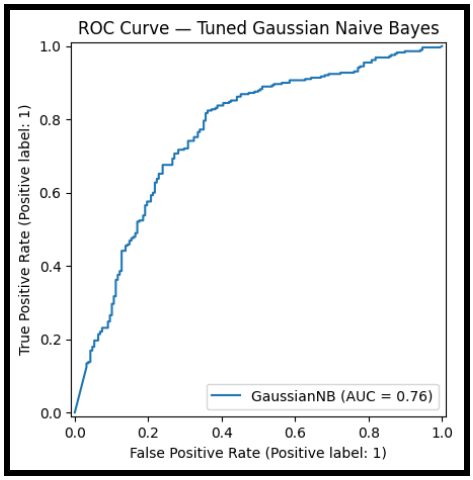
- The model is very strong at identifying retained customers (high recall).
- It produced more false positives when predicting retention.
- Performance is solid given how simple and fast the model is.

```
[[111  77]
 [ 45 245]]
              precision    recall  f1-score   support

           0      0.712     0.590     0.645       188
           1      0.761     0.845     0.801       290

    accuracy                          0.745       478
   macro avg      0.736     0.718     0.723       478
weighted avg      0.741     0.745     0.740       478
```

Metrics comparison table (baseline vs tuned)



ROC Curve (tuned model)

Model Comparisons

Best Model Selection

To figure out the most effective predictive approach for student retention, we will now compare the three classification models (KKN, tuned tree-based model using XGBoost, and a Gaussian Naive Bayes model.) Model performance is assessed using accuracy, precision, recall, and Area Under the ROC Curve, also known as AUC, in order to get predictive strength and classification error patterns.

The tuned KKN model (k=16) produced an accuracy of 0.651, precision of 0.689, recall of 0.772, and an AUC of 0.668. The recall was moderately strong, showing the model's relative ability to correctly identify retained students, but its lower precision suggests a high rate of false positives: some students predicted to be retained were actually "churn risks." Additionally, the overall accuracy was the lowest among the three models, indicating weaker general classification performance. The confusion matrix supports this, showing misclassification in both churned and retained categories.

The tuned XGBoost model showed improved performance, in comparison, across most metrics. Its best configuration achieved an accuracy of .670, precision of 0.696, recall of 0.807, and an AUC of 0.683. Compared to KNN, XGBoost showed stronger recall and slightly higher precision, depicting a better balance between identifying retained students and limiting false retention predictions. This is because the model's tree-based structure allows it to capture nonlinear relationships and interaction effects among predictors. However, its accuracy and AUC remained below the Naive Bayes model.

The tuned Gaussian Naive Bayes model was the top performer. Using optimized variance smoothing and probability tuning, the model achieved an accuracy of 0.745, precision of 0.761, recall of 0.845, and an AUC of 0.763. This accounts for a 7.5 percent-point gain in accuracy relative to the next best model. The high recall rate indicates that the model is good at identifying students who will be retained which in turn minimizes false negatives. Although precision is slightly lower than recall, the balance remains strong, as reflected in an F1 score above 0.80 after optimization.

For errors, KKN produced the most evenly distributed misclassifications. XGBoost reduced false negatives but still had a large number of false positives. Naive Bayes, adversely, showed the clearest class

separation, with fewer churned students incorrectly labeled as retained and the higher true-positive retention identification rate. A high AUC shows a stronger ability to distinguish between retained and churned students about classification thresholds.

In conclusion, while the distance and tree-based methods offered moderate predictive capabilities, the probabilistic model proved best for the distributional character of this dataset.

Model	Accuracy	Precision	Recall	AUC
KKN (k=4)	0.6674	0.7266	0.7241	0.7037
XGBoost	0.670	0.696	0.807	0.683
GNB	0.7322	0.734	0.876	0.763

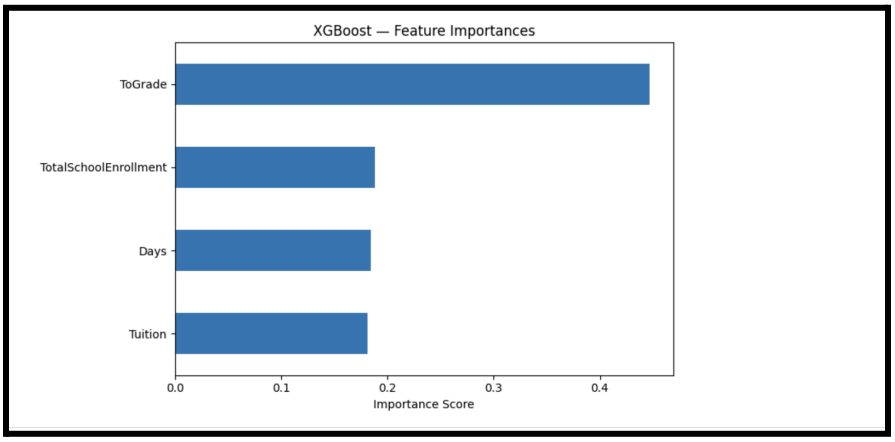
Table summary of values

Business Recommendations

Although our final predictive model selected was Naive Bayes, it doesn't produce a feature importance measure. Thus, XGBoost's feature importance plot was used as our tool to identify the predictors most strongly associated with retention outcomes. This is because tree-based ensemble models show nonlinear relationships and interactions.

Using the XGBoost's model's feature importance:

1) ToGrade (Most Important)
2) TotalSchoolEnrollment
3) Days (Trip Length)
4) Tuition



Recommendations:

- **Target retention efforts by grade level:** *ToGrade* was by far the most important predictor in the XGBoost model at a ~0.45 importance score. STC should analyze which grade levels show the highest historical retention rates and prioritize outreach and relationship-building with them before contract renewal.
- **Tailor program offerings by school size:** *TotalSchoolEnrollment* was the second strongest predictor in XGBoost. STC should prioritize renewal outreach toward larger schools, which may have more resources to coordinate repeat trips (and potentially, less flexibility 'shopping' for alternative camps with competitors).
- **Develop differentiated strategies for new vs. existing customers:** Though not captured by XGBoost, EDA showed *SPRNewExisting* had the strongest retention signal among all raw data

(~35% for new and 73% for existing). To boost their low retention rates, new customers should receive dedicated onboarding support and promotional offers.

- **Build pricing tier alternatives for price-sensitive schools:** Tuition was a consistent predictor of retention across EDA and XGBoost modeling. Offering tiered pricing or flexible payment plans could improve retention for price-sensitive schools.
- **Promote single-grade trip options:** EDA showed single grade trips retained at 80% vs. 36% for multi-grade trips. STC should actively promote single grade formats.

Limitations

XGBoost analysis uncovered a set of four highly correlated variables (*ToGrade, TotalSchoolEnrollment, Days, Tuition*), however strong predictors from EDA including *SPRNewExisting* and *IsNonAnnual* were not captured. This suggests that feature importance depends on model selection and encourages STC to consider the outputs of multiple models in aggregate, without overly indexing to a single model.

Naive Bayed, despite being the top performing model, assumes full independence between all predictors. This is an unlikely assumption to hold given the correlations between variables observed during EDA analysis.